Assignment #2 *Due:* Oct 12 by 11:55pm

Exercise 1: *Training a Perceptron to Approximate a Quadratic Function* ..... [10 marks]

In this exercise, you will simulate the training of a simple *perceptron* (a very basic model from AI to approximate the value of a quadratic function at a specific point.

Background: A perceptron with one input works like this:

$$y = w \cdot x + b$$

- w is called the *weight*,
- b is called the *bias*,
- y is the perceptron's prediction for input x.

We want the perceptron to learn the quadratic function: $f(x) = 2x^2 - 4$ at the point $x = 3$. The exact target value is: $f(3) = 2(3^2) - 4 = 14$

(a) Write a C++ program `perceptron.cpp` that trains the perceptron to approximate this target value by repeatedly adjusting its weight w and bias b.

Training Process:

1. Starts with $w = 0.0$ and $b = 0.0$.
2. Uses a *loop* to train the perceptron for 1000 steps. At each step:
   - Compute the perceptron output: $y = w \cdot x + b$ (with $x = 3$).
   - Compute the error: $error = target - y$ (with $target = 14$).
   - Update the weight and bias using a small learning rate ($\alpha = 0.01$):

$$w \leftarrow w + \alpha \cdot error \cdot x, \quad \text{and} \quad b \leftarrow b + \alpha \cdot error$$

3. Prints progress (prediction and error). To avoid too much output, print only every 100th step.
4. At the end, prints: (i) The final prediction, (ii) The target value (14), and (iii) The final values of w and b.

Sample Output (example)

```
Step 0: prediction = 0.00, error = 14.00
Step 100: prediction = 8.27, error = 5.73
Step 200: prediction = 11.94, error = 2.06
Step 300: prediction = 13.45, error = 0.55
...
Final prediction at x=3: 13.97
Target value: 14.00
Final w = 4.65, b = 0.10
```

Hints:

- Start with `double w = 0.0;` and `double b = 0.0;`
- Use `double learningRate = 0.01;`
- Check with `if (i % 100 == 0)` to print only every 100th step, where `i` is the loop variable.

(b) *Testing:* Write `perceptron-test.cpp` to test whether your trained perceptron can generalize and how well it predicts values of the quadratic function at new inputs. Take ten integer inputs from the user and reports the prediction accuracy.

- *User Input:* Ask the user to enter ten integer values of x (for example: $0, 1, 2, 3, \dots$).
- *Predictions:* For each input x_i:
  - Compute the perceptron's prediction $y_i = w \cdot x_i + b$.
  - Compute the true target $t_i = 2x^2 - 4$.
- *Compare:* A prediction is considered correct if $|y_i - t_i| \leq 1.0$ (that is, the perceptron's guess is within 1 unit of the true value).
- *Accuracy:* Count how many of the ten predictions are correct and print $accuracy = \frac{\text{correct count}}{10} \times 100\%$.
- *Output Format:* For each test point, display: x_i, y_i, t_i, "correct" or "incorrect". After all ten points, print the overall accuracy.

Exercise 2: *Plotting a function in Terminal* ..... [10 marks]

Write a C++ program that plots the function $y = \sin(x)$ in the terminal window. The program should take input the range of x values in radians over which to plot the sine wave (e.g., from -5 to 5). The output should be displayed on a 80-by-20 grid of characters, with the x-axis and y-axis clearly marked. A horizontal axis must be represented by the character `'-'` at the vertical midpoint, and a vertical axis by the character `'|'` at the horizontal midpoint. The sine wave itself should be represented by a distinct symbol such as `'*'`.

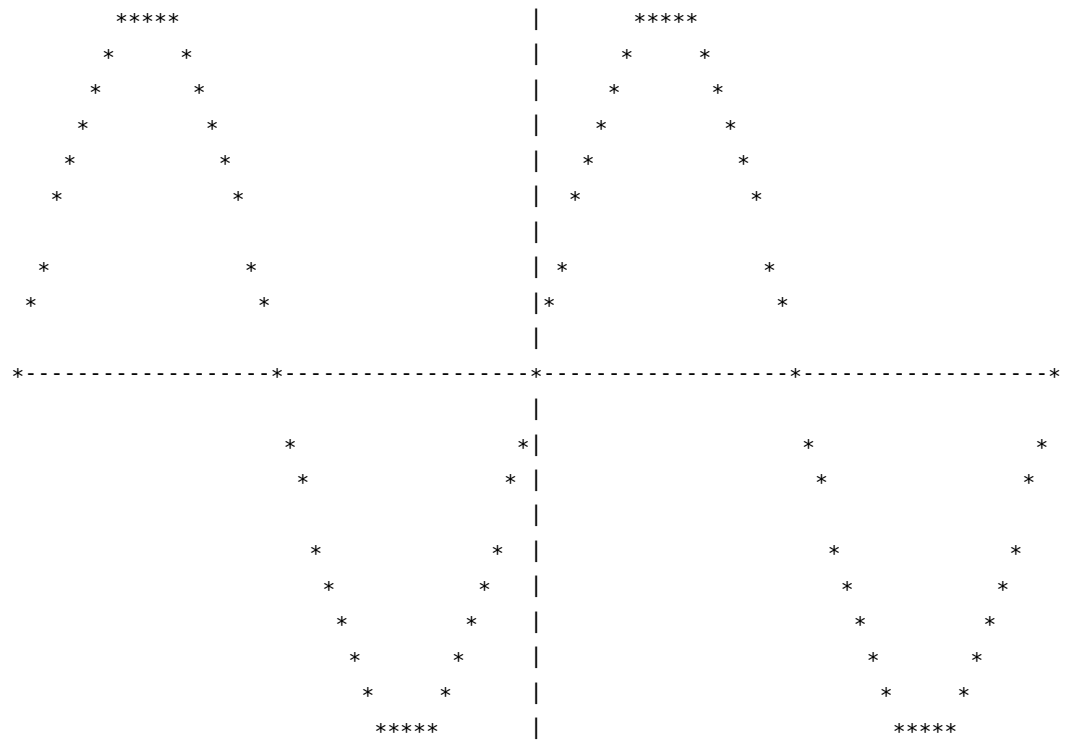
- To create the grid, you can use a 2D array of `char`s of 20 rows and 80 columns, or an 1D array of 20 `std::string`s each of length 80. . Initialize the grid with spaces and then fill in the axes and sine wave points.
- To calculate the sine values, you can use the `sin()` function from the `<cmath>` library. Remember that this function takes input in radians.
- To determine the position of each point on the grid, you will need to map the continuous x and y values to discrete grid coordinates. For this, you can use the following formulas:

$$\text{grid\_x} = \left(\frac{x - x_{\min}}{x_{\max} - x_{\min}} \right) \times (\text{grid\_width} - 1)$$

$$\text{grid\_y} = \left(\frac{y - y_{\min}}{y_{\max} - y_{\min}} \right) \times (\text{grid\_height} - 1)$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum x values, $y_{\min}$ and $y_{\max}$ are the minimum and maximum y values (for $\sin(x)$, these are -1 and 1 respectively), and grid_width and grid_height are the dimensions of your grid (80 and 20 in this case).

Sample Output ($-2\pi \leq x \leq +2\pi$)



Exercise 3: *Epidemic Evolution Simulation* ..... [10 marks]

Write a C++ program that simulates the spread of an epidemic in a population over a series of days. Individuals may recover and develop immunity, worsen to severe illness, or die. Over time, a disease may evolve and become more dangerous, making worsening outcomes increasingly likely. Each individual can be in one of the following states, represented by an integer:

- *Healthy* (0): Susceptible to infection.
- *Infected* (1–9): Actively infected with a severity value. Larger numbers indicate more severe illness.
- *Recovered* (-1): Fully immune and no longer susceptible.
- *Dead* (10): Permanently deceased.

The simulation should follow these rules:

- Each day, the state of each individual in a $W \times H$ grid is updated based on their current state and the states of their neighbors. Adjacency is defined as the four directly neighboring cells (up, down, left, right).
- A healthy individual has a probability p of becoming infected if at least one of its neighbors is infected.
  - A newly infected individual receives a random severity between 1 and the infector's severity.
  - If multiple infected neighbors target the same healthy individual, the highest resulting severity is applied.
- Infected individuals recover after R days and become immune (Recovered).
- Infected individuals evolve in severity each day:
  - With probability w, severity increases by 1.
  - With probability $1 - w$, severity decreases by 1 (simulating gradual recovery).
  - If severity reaches 9 and increases again, the individual dies (state 10).
- Recovered individuals remain immune and cannot be infected again.
- Dead individuals remain dead and do not change state.
- The simulation runs for T days, updating the state of each individual in the grid each day.

The program should take as input:

- Grid size $W \times H$ (e.g., 20×20),
- Initial number of infected individuals $0 < I < W \times H$ (randomly placed, with no overlap),
- Infection probability $0 \leq p \leq 1$,
- Worsening probability $0 \leq w \leq 1$,

while the following parameters may use default values:

- Recovery time $R = 5$ days,
- Simulation length $T = 30$ days.

Print summary statistics at the end of each day, such as the total number of healthy, infected, recovered, and dead individuals. Also, output the initial and final state of the grid after T days, using different characters to represent each state: `.` for Healthy, `1-9` for Infected with severity, `R` for Recovered, `X` for Dead.

Sample Output ($-2\pi \leq x \leq +2\pi$)

```
Enter grid width and height: 80 20
Enter initial number of infected individuals: 50
Enter infection probability p (0..1): 0.5
Enter worsening probability w (0..1): 0.5

Initial grid state:
.........................................................4......................
........2.......................................................................
....2...9..............3..................................................65....
                             ⋮
...............2..............................9...............................5.
................................................4......................9........
...........................................9.......9.........9.......5........4.

Day 1: Healthy: 1479, Infected: 116, Recovered: 3, Dead: 2
Day 2: Healthy: 1359, Infected: 222, Recovered: 15, Dead: 4
Day 3: Healthy: 1213, Infected: 333, Recovered: 47, Dead: 7
                             ⋮
Day 28: Healthy: 17, Infected: 8, Recovered: 1562, Dead: 13
Day 29: Healthy: 17, Infected: 5, Recovered: 1565, Dead: 13
Day 30: Healthy: 17, Infected: 3, Recovered: 1567, Dead: 13

Final grid state:
RRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRR..
RRRRRRRRRRRRRRRRRRRR.RRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRR
RRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRR.RRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRR
                             ⋮
RRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRR
.RRRRRRRRRRRRRRRRRRR3RRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRXRRRRRRRRR
RRRRRRRR.RRRRRRRRRRR43RR.RRRRRRRRRRRRRRRRRRXRRRRRRRXRRRRRRRRRXRRRRRRRRXRRRRRRRRR
```

Hints:

- Represent the grid as a 2D array of `int`s, where each integer corresponds to the state of an individual.
- Use a random number generator (`rand()`) to determine if a susceptible individual becomes infected or if severity worsens/improves.
- To simplify the implementation, handle 1 day first, then extend to multiple days by wrapping the logic in a loop.
- To update the grid each day, create a temporary grid to store new states before copying them back.
- Handle edge cases carefully, such as individuals on the borders of the grid.